05_01: Bibliometric Data Visualization Pipeline

Supplementary Document: Bibliometric Data Pipeline for Insider Threat Detection (SLR)

1. Overview

This document details the technical methodology used to generate the bibliometric visualizations for our Systematic Literature Review (SLR) of 110 primary studies. To ensure maximum academic rigor and reproducibility, the authors utilized a hybrid manual-computational pipeline. While keyword extraction was performed manually to ensure technical accuracy and context, the subsequent data synthesis and graphical rendering were executed using a specialized Python-based visualization suite

2. The Visualization Workflow

The pipeline followed a three-stage process:

1. **Manual Keyword Extraction:** High-density thematic auditing was performed by manually extracting keywords from the "Methodology" and "Results" sections of all 110 studies into a Microsoft Word technical log.
2. **Frequency Synthesis:** The manually extracted terms were consolidated into a Python dictionary, mapping each term to its occurrence frequency across the study corpus (N=110).
3. **Algorithmic Rendering:** The data was processed through a Python environment (Matplotlib, WordCloud, and Squarify libraries) to generate high-resolution, publication-ready figures.
4. **Layout and Finalization:** The algorithmically generated figures were subsequently imported into MS PowerPoint strictly for structural layout, label formatting, and high-resolution export to meet journal publication standards.

3. Technical Implementation (Source Code)

3.1. Word Cloud & Bar Chart Generation

The Word Cloud was configured to visualize thematic density. The Bar Chart focuses on the most significant research terms identified in the literature.

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud
# Thematic Dictionary based on the 110-study corpus
keywords_freq = {
  'Insider Threat Detection': 110, 'Anomaly Detection': 65,
  'Machine Learning': 55, 'Deep Learning': 50, 'Cybersecurity': 45,
  'User Behavior Analysis': 40, 'Behavioral Analysis': 35,
  'Graph Neural Networks': 28, 'NLP / Sentiment Analysis': 25,
  'Data Imbalance Handling': 22, 'Network Security': 20,
  'Federated Learning': 18, 'Ensemble Learning': 14,
  'Zero Trust Architecture': 12, 'XAI': 7, 'Explainable AI': 7
  # ... [Complete dictionary available in Master_Data.xlsx]
}
# Dense Word Cloud Rendering
wc = WordCloud(
  width=1600, height=1000, background_color='white', colormap='plasma',
  max_words=600, random_state=42, relative_scaling=0.25
).generate_from_frequencies(keywords_freq)

plt.figure(figsize=(20, 12))
plt.imshow(wc, interpolation='bilinear')
plt.axis("off")
plt.tight_layout(pad=0)
plt.show()
```

3.2. Keyword Tree Map

To understand the specialized focus of the literature, a Tree Map was generated to show the Top 10 Key terms found in the literature

```
import squarify

data = {
  'Keyword': ['Insider Threat Detection', 'Anomaly Detection', 'Machine Learning',
        'Deep Learning', 'Cybersecurity', 'User Behavior Analysis',
        'Behavioral Analysis', 'Graph Neural Networks',
        'NLP / Sentiment Analysis', 'Data Imbalance Handling'],
  'Count': [110, 65, 55, 50, 45, 40, 35, 28, 25, 22]
}

# Formatting for large-font academic readability
squarify.plot(sizes=data['Count'], label=labels, color=plt.cm.Set2.colors, alpha=1.0)
plt.axis('off')
plt.show()
```

3.3. Publication Trend & Journal Distribution

These scripts track the publication volume from 2021–2025 and identify the leading venues (e.g., *IEEE Access*, *Computers & Security*).

```
import matplotlib.pyplot as plt
from google.colab import files

# 1. Exact Counts from your 110-study list
years = ['2021', '2022', '2023', '2024', '2025']
counts = [15, 13, 26, 25, 31]

# 2. Create the Visualization
plt.figure(figsize=(10, 6))

# Create the bars with a clean professional blue
bars = plt.bar(years, counts, color='#3498DB', alpha=0.85,
label='Number of Studies')

# Add a trend line to highlight the growth trajectory
plt.plot(years, counts, color='#E74C3C', marker='o', markersize=8,
linewidth=2.5, label='Growth Trend')

# 3. Styling & Aesthetics
plt.xlabel('Year of Publication', fontsize=12, fontweight='bold')
plt.ylabel('Number of Studies', fontsize=12, fontweight='bold')
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.ylim(0, max(counts) + 5)
plt.legend(frameon=True, loc='upper left')

# Remove top and right spines for a modern academic look
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
# Add exact count labels on top of each bar
for bar in bars:
  yval = bar.get_height()
  plt.text(bar.get_x() + bar.get_width()/2, yval + 0.8, int(yval),
      ha='center', va='bottom', fontsize=11, fontweight='bold')

plt.tight_layout()
# 4. SAVE AND DOWNLOAD
filename = "SLR_publication_trend_final.png"
plt.savefig(filename, dpi=300, bbox_inches='tight')
print(f"Graph generated with {sum(counts)} papers.
Downloading...")
plt.show()
files.download(filename)
```

```
import matplotlib.pyplot as plt
from google.colab import files
# 1. Prepare the Data (Top 10 for clarity)
journals = [
  'IEEE Access',
  'Computers & Security',
  'IEEE Trans. on Information Forensics & Security',
  'Scientific Reports',
  'Int. Journal of Adv. Computer Science & Apps',
  'Future Internet',
  'Computers & Electrical Engineering',
  'Journal of Info. Security & Applications',
  'Future Generation Computer Systems',
  'IEEE Trans. on Dependable & Secure Computing'
]
counts = [11, 6, 5, 4, 4, 3, 3, 3, 2, 2]
# 2. Create the Visualization
plt.figure(figsize=(12, 7))
# Sort the data so the highest is at the top
journals.reverse()
counts.reverse()
# Horizontal bar chart with a professional dark teal color
bars = plt.barh(journals, counts, color='#2471A3', alpha=0.9)

# 3. Aesthetics & Styling
plt.xlabel('Number of Publications', fontsize=12,
fontweight='bold')
plt.grid(axis='x', linestyle='--', alpha=0.5)

# Removing spines for a cleaner academic look
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)

# Add exact count labels at the end of each bar
for i, v in enumerate(counts):
  plt.text(v + 0.2, i, str(v), color='black', va='center',
fontweight='bold', fontsize=11)
plt.tight_layout()
# 4. SAVE AND DOWNLOAD
filename = "top_journals_analysis.png"
plt.savefig(filename, dpi=300, bbox_inches='tight')
print("Journal analysis graph generated! Downloading...")
plt.show()
files.download(filename)
```